Implementação de Mecanismos de QoS usando VNFs: Proposta de Projeto

GRUPO 6

Ivo Costa Sousa – pg63976

João Afonso Almeida Sousa – pg62405

João Carlos Teixeira Neiva – pg63978

Novembro de 2026

Resumo

Esta proposta, no âmbito do Trabalho Prático 3 da Unidade Curricular de Qualidade de Serviço na Internet, descreve um sistema de QoS baseado em Funções de Rede Virtualizadas (VNFs): uma cadeia de três VNFs no caminho de dados (controlo de acesso, classificação e controlo de tráfego) e uma VNF de monitorização, emuladas em Containernet sobre uma topologia semelhante à de um ISP. Apresentam-se a topologia, a política DiffServ, as ferramentas, os cenários de teste e o planeamento.

1 Objetivos

O projeto cobre os cinco aspetos de QoS do enunciado (classificação, controlo de acesso, policiamento, agendamento e monitorização), cada um implementado como VNF num contentor Docker. Com a ligação de acesso de 10 Mbit/s congestionada, a cadeia deve garantir:

- **VoIP:** atraso num sentido inferior a 150 ms (ITU-T G.114 [7]), *jitter* inferior a 30 ms e perdas inferiores a 1%.
- **Vídeo:** débito de pelo menos 4 Mbit/s com perdas inferiores a 1%.
- **Best effort:** utilização da capacidade restante sem esgotar as outras classes.
- **Tráfego não autorizado:** 100% descartado na primeira VNF da cadeia.

Cada meta é medida com as VNFs em modo de simples encaminhamento (sem QoS) e com a cadeia ativa, de forma a quantificar o ganho e o custo de cada função.

2 Topologia

A topologia imita um ISP: a LAN de clientes chega à LAN de servidores através da cadeia de VNFs, de uma ligação de acesso de 10 Mbit/s e de dois *routers* de borda (Figura 1). Todos os nós são contentores Docker criados pelo *script* `topologia.py`, que também configura endereços, rotas e as características das ligações (`tbf` no acesso e `netem` no núcleo). O plano de endereçamento completo consta do `README` do repositório.

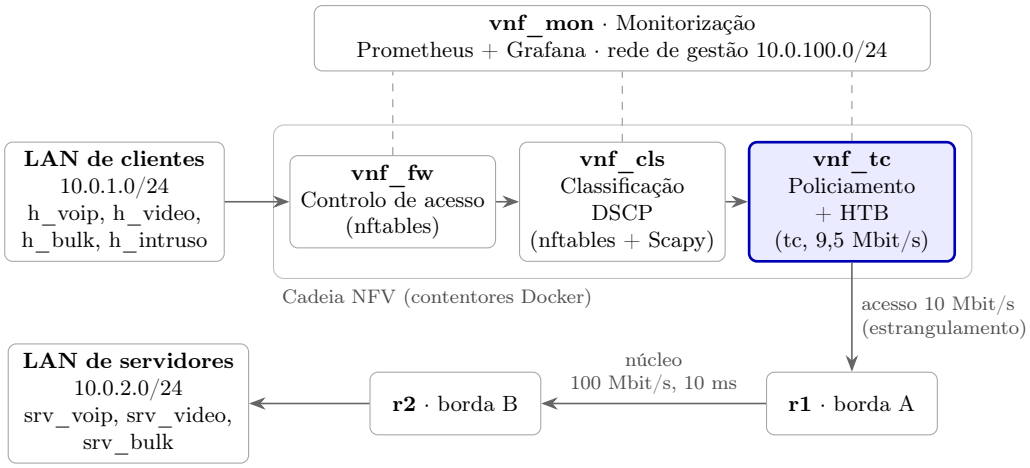


Figura 1: Topologia em Containernet. A tracejado, a rede de gestão; a destaque, a VNF onde se forma a fila em congestionamento.

3 Arquitetura de QoS

A cadeia de serviço é **vnf_fw → vnf_cls → vnf_tc**, com a vnf_mon fora do caminho de dados. Cada VNF é um salto IP com encaminhamento ativo, encadeado por rotas estáticas. A ordem difere do exemplo do enunciado. O controlo de acesso vem primeiro para que o tráfego não autorizado seja descartado antes de consumir CPU nas VNFs seguintes. A classificação vem antes do controlo de tráfego porque o policiamento e as filas decidem pela marca DSCP, que já tem de estar no pacote quando ele lá chega. A monitorização fica fora do caminho de dados: recolhe os contadores pela rede de gestão e não acrescenta atraso aos pacotes.

A política segue o modelo DiffServ [1, 2] com três classes (Tabela 1). A vnf_tc limita a saída a 9,5 Mbit/s, abaixo dos 10 Mbit/s da ligação de acesso, para que a fila se forme dentro da VNF, onde o agendador decide, e não no enlace. O controlo de acesso usa uma lista branca com estado: só passa tráfego da LAN de clientes para a LAN de servidores nas portas da tabela, ICMP e respostas a ligações estabelecidas; tudo o resto, incluindo o h_intruso, é descartado e contado.

Tabela 1: Política DiffServ aplicada pela cadeia

Classe	**Tráfego**	**Critério (vnf_cls)**	**DSCP**	**Tratamento (vnf_tc)**
VoIP	UDP 10000–10099 (RTP)	porta e tamanho < 300 B	EF (46) [3]	fila prioritária, policiada a 1 Mbit/s (*token bucket*, rajada 10 kB)
Vídeo	UDP 5004	porta	AF41 (34) [4]	HTB: 5 Mbit/s garantidos, teto 9 Mbit/s
Best effort	TCP 5201 e restante	por omissão	BE (0)	HTB: 2 Mbit/s garantidos, teto 9,5 Mbit/s, fq_codel [5]

4 Ferramentas

Todas as VNFs correm como contentores Docker a partir de uma imagem comum (Ubuntu 22.04), com o repositório montado em `/tp3`. O ambiente é o Containernet [8] (Mininet, Open vSwitch e Docker) numa VM em VirtualBox, e a separação das funções em VNFs segue a arquitetura de referência ETSI NFV [6]. O tráfego é gerado com D-ITG [9] (VoIP, com atraso num sentido) e iperf3 (vídeo e dados), e a degradação do núcleo com `tc netem`. O controlo de versões é feito em Git.

- `vnf_fw`: nftables com estado e contadores por regra.

- `vnf_cls`: nftables para marcação DSCP no caminho rápido; Python com Scapy e NetfilterQueue para inspeção RTP, comparada em débito e latência.
- `vnf_tc`: `tc` com *police* (*token bucket*) na classe EF e HTB com fq_codel; PRIO como alternativa.
- `vnf_mon`: Prometheus e Grafana, com node_exporter e um *exporter* em Python para `tc -s class` e contadores do nftables.

Como extensões para bónus, se o calendário o permitir, prevê-se a classificação por aprendizagem automática (árvore de decisão sobre o tamanho e o intervalo entre pacotes) e o encaminhamento da cadeia por regras OpenFlow no Open vSwitch, em vez de rotas estáticas.

5 Cenários de Tráfego e Experiências

Quatro fluxos, que somam mais de 10 Mbit/s, garantem congestionamento na ligação de acesso (Tabela 2). O iperf3 abre ainda uma ligação TCP de controlo na mesma porta, que a firewall terá de permitir.

Tabela 2: Fluxos de tráfego

Fluxo	**Origem → destino**	**Ferramenta**	**Perfil**	**Métricas**
VoIP	h_voip → srv_voip	D-ITG (G.711, RTP)	50 pacotes/s, ≈80 kbit/s, UDP 10000	atraso, *jitter*, perdas
Vídeo	h_video → srv_video	iperf3 UDP	4 Mbit/s constante, pacotes de 1316 B, UDP 5004	débito, *jitter*, perdas
Dados	h_bulk → srv_bulk	iperf3 TCP	4 fluxos paralelos, saturante, TCP 5201	débito, retransmissões
Intruso	h_intruso → srv_bulk	iperf3 UDP, nc	20 Mbit/s para UDP 5202; varrimento de portas 20–30	pacotes bloqueados, efeito nos restantes

As experiências combinam estes fluxos (Tabela 3). Cada uma dura 60 s e repete-se 5 vezes, reportando-se a média e o intervalo de confiança a 95%. Mede-se também o custo da própria cadeia: atraso acrescentado por cada VNF e CPU consumido por contentor.

Tabela 3: Plano de experiências

Id	**Mistura**	**Estado da cadeia**	**Pergunta**
E0	VoIP e ping	sem QoS	Linha de base sem carga.
E1	VoIP, vídeo e dados	sem QoS	Quanto sofre o VoIP com congestionamento?
E2	VoIP, vídeo e dados	com QoS	As metas da Secção 1 são cumpridas?
E3	E2 com `netem` no núcleo (perdas 1%, atraso 50 ± 10 ms)	com QoS	A cadeia aguenta condições de Internet?
E4	E2 com intruso	sem e com *firewall*	O controlo de acesso protege as outras classes?
E5	E2	PRIO vs. HTB, vários pesos	Que agendador e pesos equilibram melhor as classes?

6 Planeamento

O projeto ocupa sete semanas, de 30 de outubro a 17 de dezembro de 2026, com três entregas: proposta a 12 de novembro, relatório de progresso a 3 de dezembro e entrega final a 17 de dezembro (Figura 2). O enunciado indica a proposta para o fim da semana 2 na Fase 1 e para a semana 3 na lista de entregáveis; o plano assume a data mais cedo.

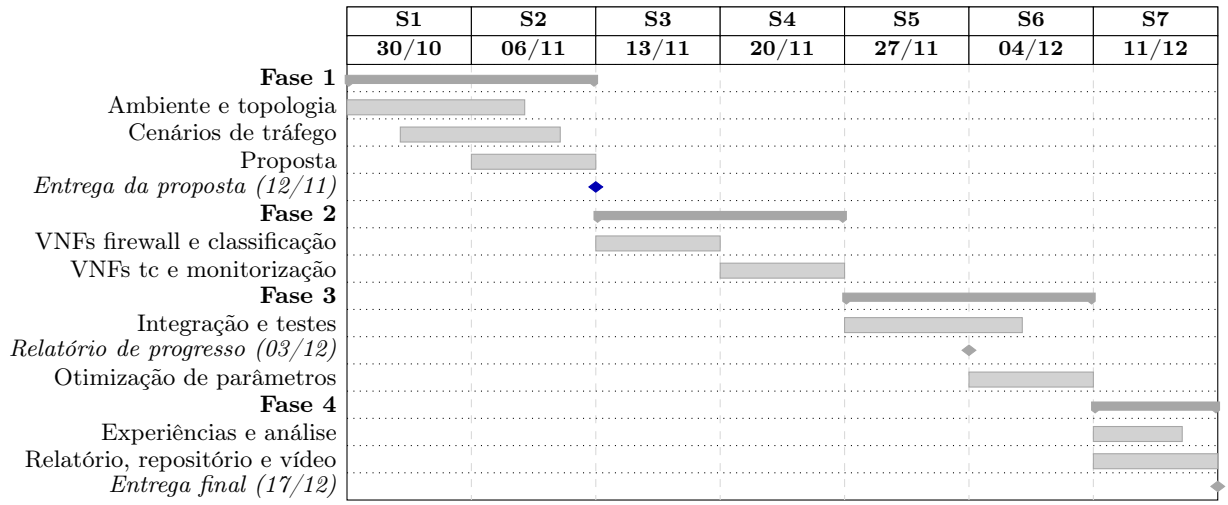


Figura 2: Diagrama de Gantt do projeto (semanas 1 a 7).

7 Riscos e Mitigação

Se o Containernet falhar com a versão do *kernel*, o plano B é Mininet simples com as VNFs como processos. Como o Scapy processa apenas alguns milhares de pacotes/s, a marcação DSCP fica em nftables e o Scapy é reservado à inspeção RTP. Os *offloads* (TSO, GSO, GRO), que falseiam o `tc` em interfaces virtuais, são desligados com `ethtool`. Por fim, o atraso num sentido é fiável porque todos os contentores partilham o relógio da mesma VM.

Referências

[1] S. Blake *et al.*, "An Architecture for Differentiated Services," RFC 2475, IETF, 1998.

[2] K. Nichols *et al.*, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers," RFC 2474, IETF, 1998.

[3] B. Davie *et al.*, "An Expedited Forwarding PHB (Per-Hop Behavior)," RFC 3246, IETF, 2002.

[4] J. Heinanen *et al.*, "Assured Forwarding PHB Group," RFC 2597, IETF, 1999.

[5] T. Høiland-Jørgensen *et al.*, "The Flow Queue CoDel Packet Scheduler and Active Queue Management Algorithm," RFC 8290, IETF, 2018.

[6] ETSI, "Network Functions Virtualisation (NFV); Architectural Framework," ETSI GS NFV 002 V1.2.1, 2014.

[7] ITU-T, "One-way transmission time," Recomendação G.114, 2003.

[8] M. Peuster, H. Karl e S. van Rossem, "MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments," em *Proc. IEEE NFV-SDN*, 2016.

[9] A. Botta, A. Dainotti e A. Pescapè, "A tool for the generation of realistic network workload for emerging networking scenarios," *Computer Networks*, vol. 56, n.º 15, 2012.